UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ
COORDENADORIA DO CURSO DE ENGENHARIA DE SOFTWARE

RAFAEL BERNARDY DE SOUZA

GATEWAY DE AUTENTICAÇÃO, CUSTÓDIA E CONSUMO DE API

TRABALHO DE CONCLUSÃO DE CURSO 2

DOIS VIZINHOS

2026

RAFAEL BERNARDY DE SOUZA

GATEWAY DE AUTENTICAÇÃO, CUSTÓDIA E CONSUMO DE API

Trabalho de Conclusão de Curso 2 apresentado como requisito parcial para obtenção do título de Bacharel em Engenharia de Software da Universidade Tecnológica Federal do Paraná (UTFPR), Câmpus Dois Vizinhos.

Orientador: Newton Carlos Will

Co-orientador: Erinaldo da Silva Pereira

DOIS VIZINHOS

2026

SUMÁRIO

RESUMO .....
ABSTRACT .....
1 INTRODUÇÃO **5**
2 PROJETO E IMPLEMENTAÇÃO **6**
2.1 AUTENTICAÇÃO E CONSUMO 6
2.2 FATURAS E COMPONENTES DE LABORATÓRIO 7
3 VALIDAÇÃO REALIZADA **8**
3.1 TESTES AUTOMATIZADOS 8
3.2 ENSAIOS DE FALHA E PAGAMENTO PONTUAL 8
4 DISCUSSÃO E LIMITES **9**
5 CONSIDERAÇÕES FINAIS **10**
REFERÊNCIAS **11**

RESUMO

BERNARDY DE SOUZA, Rafael. GATEWAY DE AUTENTICAÇÃO, CUSTÓDIA E CONSUMO DE API. 11 f. Trabalho de Conclusão de Curso 2 – Coordenadoria do Curso de Engenharia de Software, Universidade Tecnológica Federal do Paraná. Dois Vizinhos, 2026.

Este trabalho investiga a consistência transacional em um gateway de API tarifado em satoshis, no qual a liquidação ocorre em rede externa (Lightning Network) e o controle de saldo e métricas reside em banco relacional local. Desenvolveu-se um protótipo em FastAPI com autenticação por assinaturas Ed25519 sob desafio-resposta, tarifação idempotente (por requisição, volume de bytes ou contagem de tokens) e persistência em SQLite e PostgreSQL. Foram implementados módulos experimentais para integração REST com LNbits, validação de faturas BOLT11, verificação do protocolo LNURL-Pay (LUD-06), particionamento de eventos para TimescaleDB e controle de emissão via outbox local. A suíte automatizada totaliza 76 testes cobrindo concorrência, replay e tolerância a falhas de comunicação. O protótipo demonstra a viabilidade do controle transacional e da tarifação granular, delimitando os pontos em que estados de rede ambíguos exigem reconciliação assistida.

Palavras-chave: Gateway de API, Lightning Network, Ed25519, idempotência, PostgreSQL

ABSTRACT

BERNARDY DE SOUZA, Rafael. API Gateway for Authentication, Custody, and Usage-Based Billing. 11 f. Trabalho de Conclusão de Curso 2 – Coordenadoria do Curso de Engenharia de Software, Universidade Tecnológica Federal do Paraná. Dois Vizinhos, 2026.

This work investigates transactional consistency in an API gateway billed in satoshis, where payment settlement occurs on an external network (Lightning Network) while balance and metric management resides in a local relational database. A FastAPI prototype was developed with Ed25519 challenge-response authentication, idempotent metering (by request, byte volume, or token count), and transactional persistence in SQLite and PostgreSQL. Experimental modules were implemented for LNbits REST integration, BOLT11 invoice validation, LNURL-Pay protocol verification (LUD-06), event partitioning for TimescaleDB, and an outbox queue for local issuance control. The automated test suite contains 76 test cases covering concurrency, replay prevention, and communication failure handling. The prototype demonstrates the feasibility of transactional control and granular billing, identifying the boundaries where ambiguous network states require assisted reconciliation.

Keywords: API gateway, Lightning Network, Ed25519, idempotency, PostgreSQL

1 INTRODUÇÃO

A proposta de TCC 1 descreveu um gateway para autenticação de clientes e cobrança por requisição, byte ou token com créditos vinculados à Lightning Network, originalmente descrita por Poon e Dryja (POON; DRYJA, 2016). A principal dificuldade de implementação reside na coordenação entre o estado interno do gateway e o serviço externo de liquidação. Em caso de perda de resposta ou timeout na emissão de uma fatura, a ausência de confirmação impede inferir se ela foi gerada pelo provedor. Uma nova tentativa pode duplicar a emissão, enquanto o descarte prematuro inviabiliza a vinculação de um eventual pagamento à conta do usuário.

O escopo deste trabalho concentra-se no tratamento desse intervalo de incerteza. O protótipo implementa protocolo de desafio-resposta para autenticação, garantindo idempotência contábil por meio de identificadores únicos e suspendendo reemissões automáticas sob respostas ambíguas. A arquitetura foi estruturada em duas camadas: a API principal, com faturas simuladas para validação do ciclo de débito e autenticação, e módulos isolados de bancada para verificação de interoperabilidade com LNbits, Alby e Blink.

O problema abordado consiste em assegurar que falhas de comunicação ou retransmissões não resultem em duplicidade de débitos ou créditos. Os testes automatizados e cenários de injeção de falhas avaliam o comportamento do sistema diante de concorrência e interrupções abruptas, permitindo verificar os limites dos mecanismos de idempotência propostos.

2 PROJETO E IMPLEMENTAÇÃO

2.1 AUTENTICAÇÃO E CONSUMO

O cadastro da conta ocorre por comando administrativo local. O cliente guarda sua chave privada Ed25519, esquema definido na RFC 8032 (JOSEFSSON; LIUSVAARA, 2017), e fornece ao gateway apenas a chave pública. Para cada operação, solicita em `/v1/challenges` um nonce associado à conta, ao identificador da requisição, à rota e ao SHA-256 do conteúdo esperado. O desafio expira em 60 segundos. A assinatura cobre uma mensagem canônica com esses campos.

Na execução, o gateway valida a assinatura, aplica bloqueio pessimista de linha (`SELECT ... FOR UPDATE`) no PostgreSQL para garantir exclusão mútua sobre a conta e marca o desafio como consumido mediante atualização condicional. O mesmo nonce não pode ser consumido duas vezes. O identificador de requisição permite repetição lógica com novo desafio: se conteúdo e preço forem os mesmos, a resposta retorna o débito já registrado. Caso contrário, a API rejeita o conflito. A rota `/v1/consume` ecoa o conteúdo recebido para evidenciar o processamento e suporta tarifação em três modalidades: por requisição (`request`), por volume de dados (`bytes`, em degraus de 1024 bytes) ou por tokens de texto (`tokens`, via codificador BPE `cl100k_base`). Para mitigar erros de entrada ou consumo desmedido de saldo, fixou-se um teto operacional de um milhão de satoshis por requisição.

A atualização de saldo é condicional:

```
UPDATE accounts SET balance = balance - :price
WHERE id = :account_id AND balance >= :price;
```

Se nenhuma linha for atualizada, o saldo não cobre o preço. Confirmado o débito, a atualização do saldo, a baixa do desafio, o registro contábil e o evento analítico (`usage_events`) são persistidos atomicamente em transação única. Em caso de falha na gravação analítica, a transação é revertida integralmente. A tabela `usage_events` adota chave primária composta (`account_id`, `request_id`, `occurred_at`), viabilizando o particionamento temporal para conversão em hi-

pertabela TimescaleDB sem comprometer a integridade referencial com os débitos contábeis.

2.2 FATURAS E COMPONENTES DE LABORATÓRIO

A aplicação FastAPI injeta o adaptador simulado `FakeInvoices` por padrão e rejeita adaptadores externos na rota HTTP de emissão. A confirmação de uma fatura emitida por esse dublê só se converte em crédito após consulta ao adaptador, verificação de montante e carteira de destino e lançamento atômico no banco de dados. O estado em memória do emissor simulado é volátil e não persiste após a reinicialização da aplicação, não caracterizando saldo custodial.

Há um adaptador REST para o contrato LNbits v1.6.2, mas ele não está ligado à emissão da API pública. O protocolo LNURL-Pay, especificado no LUD-06 (FIATJAF; AKUMAI-GORODSKI, s.d.), foi implementado no módulo de laboratório `OfflineLNURLPay`: ele valida metadados textuais, limites `minSendable` e `maxSendable`, protege o sistema contra redirecionamentos e SSRF ao validar a URL do callback, e confere se a fatura BOLT 11 (LIGHTNING NETWORK, s.d.) vincula o hash SHA-256 exato dos metadados na tag `h` (`description_hash`). Também há código de validação de faturas BOLT 11 que confere rede, montante, `payment_hash`, expiração e assinatura. A ativação de hipertabela TimescaleDB foi implementada no script administrativo `enable_timescale_usage.py`, voltado ao ambiente de testes e programado para abortar com mensagem informativa caso a extensão não esteja instalada no cluster PostgreSQL.

No laboratório, `fixture_outbox.py` mantém uma outbox SQLite privada para limitar emissões de teste a 10 satoshis por arquivo. Cada reserva grava `issuing` antes do POST; a validação de uma resposta permite a transição para `recorded`. Timeout, resposta inválida ou reinício durante `issuing` levam a `uncertain`. Não há transição automática de `uncertain` para nova emissão. O limite continua consumido até uma análise explícita. Essa outbox de proxy e a tabela `invoice_attempts` do gateway não formam uma transação distribuída única. A tabela de controle de emissões do gateway é exercitada exclusivamente com o emissor simulado, sem emissão remota pela API pública.

O proxy de laboratório usa socket Unix e verifica as credenciais do processo por `SO_PEERCRED`. Os filtros de rota e operação bloqueiam o envio de pagamentos nos ensaios. Essas barreiras foram testadas em condições controladas, voltadas ao isolamento de bancada.

3 VALIDAÇÃO REALIZADA

3.1 TESTES AUTOMATIZADOS

A validação do sistema fundamenta-se em uma suíte automatizada executada contra instâncias de SQLite e uma base dedicada do PostgreSQL 16. O conjunto reúne 76 casos de teste aprovados sem omissões, contemplando testes unitários, validações offline de protocolos e cenários de integração transacional. Os resultados atestam a correção lógica dos fluxos sob entradas controladas, não contemplando ensaios de estresse ou transações financeiras em larga escala.

Os cenários cobrem assinatura inválida, replay de nonce, requisições concorrentes, reversão transacional após falha ao registrar débito ou métrica de uso, tarifação determinística por bytes e tokens com limites estritos, validação de contrato LNURL-Pay LUD-06, faturas com valor divergente e tratamento de estados incertos de emissão.

3.2 ENSAIOS DE FALHA E PAGAMENTO PONTUAL

O relatório de falhas da outbox descreve quatro intervenções com upstream sintético: falha antes do POST, perda de resposta após POST, encerramento do daemon e reinício do proxy. Nas falhas sem confirmação, a reserva permaneceu na outbox e não houve reenvio automático. No cenário em que o upstream recebeu o POST e a resposta se perdeu, não há `payment_hash` para reconciliação automática: o estado é `uncertain` e requer investigação humana. Portanto, a outbox limita duplicação de emissão, mas não elimina por si só o risco de fatura órfã.

No ensaio realizado em 21/10/2026 na rede principal, uma fatura de 10 satoshis foi confirmada como `PAID` via carteira Blink. A conta de laboratório recebeu o crédito correspondente e registrou consumo autenticado de 1 satoshi, restando 9 satoshis de saldo. Trata-se de uma validação pontual de bancada para confirmação do fluxo prático, não constituindo amostra estatística de desempenho ou integração direta da API pública com o serviço LNbits.

4 DISCUSSÃO E LIMITES

A hipótese de redução de custos da ordem de 60% foi apresentada no TCC 1 como algo a investigar. Este trabalho não mediu essa redução. Uma comparação empírica requer modelar volume de chamadas, preços, taxas de rede, liquidez e alternativas de processamento sob o mesmo cenário. Também são necessários testes de carga para avaliar o impacto dos locks do PostgreSQL e das escritas na outbox SQLite sob alto paralelismo.

Identificam-se três distinções em relação ao texto da proposta inicial. Quanto à autenticação, adotou-se Ed25519 para manter as chaves de API segregadas das chaves de custódia da carteira (o TCC 1 previa esquemas Schnorr/BIP-340). No armazenamento analítico, o gateway opera em PostgreSQL convencional, preservando a compatibilidade de particionamento para quando o TimescaleDB for habilitado. Por fim, o protocolo LNURL-Pay foi validado em laboratório; a comunicação com clientes externos dependerá de infraestrutura com terminação TLS e resolução DNS pública.

Não foram implementados o provisionamento via contêineres Docker, instrumentação para observabilidade com Prometheus e Grafana, avaliação comparativa de desempenho com outros gateways ou mecanismos de recuperação de faturas sem hash de pagamento. Adicionalmente, o ensaio de liquidação pontual realizado na rede Blink teve caráter de prova de conceito controlada, não conferindo ao protótipo os requisitos operacionais e de segurança necessários para um ambiente de custódia em produção.

5 CONSIDERAÇÕES FINAIS

Este trabalho apresentou a implementação funcional de um gateway com autenticação Ed25519, consumo atômico de desafios, débito idempotente por requisição, bytes ou tokens e rastreabilidade contábil e analítica. A suíte local e os ensaios isolados permitem reproduzir falhas que seriam difíceis de observar em uma demonstração apenas de caminho feliz. O estado de incerteza operacional é o limite principal: sem uma referência externa verificável, a decisão segura do protótipo é interromper a operação e exigir reconciliação assistida.

As etapas seguintes compreendem a definição, junto aos orientadores, dos experimentos quantitativos prioritários para o período restante, em especial medições de latência e custo sob carga. Até lá, os resultados dos testes e o pagamento pontual constituem evidências dos cenários executados, delimitando o escopo real do protótipo.

REFERÊNCIAS

JOSEFSSON, Simon; LIUSVAARA, Ilari. **Edwards-Curve Digital Signature Algorithm (EdDSA)**. RFC 8032. [S. l.]: RFC Editor, jan. 2017. Disponível em: https://www.rfc-editor.org/rfc/rfc8032. Acesso em: 22 out. 2026.

LIGHTNING NETWORK. **BOLT 11: Invoice Protocol for Lightning Payments**. Repositório bolts, [s. d.]. Disponível em: https://github.com/lightning/bolts/blob/master/11-payment-encoding.md. Acesso em: 22 out. 2026.

FIATJAF; AKUMAIGORODSKI, Anton. **LUD-06: payRequest base spec**. Repositório luds, [s. d.]. Disponível em: https://github.com/lnurl/luds/blob/luds/06.md. Acesso em: 22 out. 2026.

POON, Joseph; DRYJA, Thaddeus. **The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments**. Versão preliminar 0.5.9.2, 14 jan. 2016. Disponível em: https://lightning.network/lightning-network-paper.pdf. Acesso em: 22 out. 2026.